

INFORMÁTICA

PRÁCTICA 5

Resuelve los siguientes ejercicios usando listas.

5.1. Conjetura de Ulam

La conjetura de Ulam afirma que dado un entero y siguiendo los pasos siguientes siempre obtenemos un 1.

- Si el número es par se divide por 2.
- Si es impar se multiplica por 3 y se suma 1.
- Si el resultado es 1 se para, en caso contrario se vuelve al paso a. aplicado sobre el resultado.

Escribe una función `ulam` que reciba como argumento un número entero y que devuelva una lista con todos los valores obtenidos en el algoritmo. Ejemplo:

```
>>> ulam(5)
[5, 16, 8, 4, 2, 1]
>>> ulam(7)
[7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
```

5.2. Máximos de una señal

Una señal puede representarse por una secuencia de valores reales que corresponden a los valores de la señal en instantes periódicos de tiempo. Los instantes de tiempo suelen identificarse por números enteros positivos consecutivos `[0,1,2,3,...]`.

Implementa una función `maximos(v)` que admite un único argumento `v` que codifica la señal. Debe devolver las posiciones (instantes de tiempo) en las que la señal tiene un máximo. En caso de que todos los valores de la señal sean el mismo, se devolverá una lista vacía.

```
>>> maximos([0,1,2,3,2,3,0])
[3, 5]
>>> maximos([2,2,2,2,2,2])
[]
```

5.3. Desviación estándar

La desviación estándar de una señal discreta x se define como:

$$\sqrt{\frac{1}{N-1} \sum_{i=0}^{N-1} (x_i - \mu)^2}$$

Donde μ es la media de la señal y N el número de muestras en la señal. Elabora una función `std(x)` que devuelva el valor de la desviación estándar de la señal dada como argumento de entrada. Ejemplo:

```
>>> std([2,2,1,2,3,2,1,2,2])
0.6009252125773316
>>> std([1,1,1,1,2,1,1,1,1])
0.3333333333333333
```

5.4. Normalización

La operación de normalización de una señal permite construir una señal con media 0 y desviación estándar 1 que tiene propiedades estadísticas análogas a la señal original. Para ello es preciso calcular el *z-score* de cada muestra en la señal utilizando la siguiente fórmula:

$$z = \frac{x_i - \mu}{\sigma}$$

Donde μ es la media de la señal y σ es la desviación estándar.

Elabora la función `normalize(x)` que devuelve una lista que defina la señal x normalizada. Puedes utilizar la función del ejercicio anterior. Ejemplo:

```
>>> normalize([1.1,.9,1.1,.9])
[0.8660254037844392, -0.8660254037844382, 0.8660254037844392,
-0.8660254037844382]
>>> normalize([0,2,4])
[-1.0, 0.0, 1.0]
```

5.5. Ajuste lineal por mínimos cuadrados

El ajuste lineal consiste en encontrar la recta que mejor aproxima los valores de una señal. Un método ampliamente utilizado es el ajuste por mínimos cuadrados. Este método determina que una colección de puntos (x_i, y_i) se puede aproximar por la recta $y = a x + b$ donde la pendiente, a , y el corte con la recta de abscisas, b , pueden calcularse según las fórmulas indicadas.

$$a = \frac{\sum xy - \frac{\sum x \sum y}{n}}{\sum x^2 - \frac{(\sum x)^2}{n}}$$

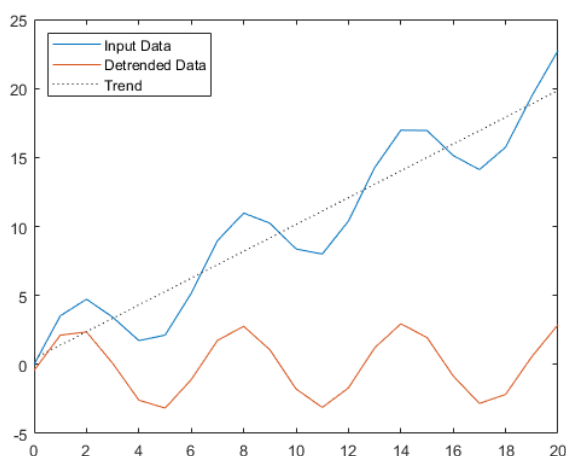
Define una función `ajuste_lineal(y)` que devuelve los valores `a` y `b` descritos previamente. Utiliza como variable independiente `x` el valor del tiempo discreto. Es decir, las posiciones de los valores de la señal en la secuencia. Ejemplos:

$$b = \frac{\sum y - a \sum x}{n}$$

```
>>> ajuste_lineal([1,1.1,.9,1,1.1,.9,1.1])
(0.003571428571428622, 1.0035714285714283)
>>> ajuste_lineal(range(5,100,2))
(2.0, 5.0)
```

5.6. Eliminación de tendencia lineal

Para analizar estadísticamente una señal con una tendencia lineal es frecuentemente necesario eliminar esta tendencia. Para ello se hace primero un ajuste lineal y se resta de la señal el valor correspondiente al ajuste lineal. Por ejemplo:



Elabora la función `detrend(y)` que devuelve una señal que elimina la tendencia lineal de la señal que se pasa como argumento. Utiliza la función del ejercicio anterior. Ejemplos:

```
>>> detrend(range(10))
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
>>> detrend([1,1.1,.9,1,1.1,.9,1.1])
[-0.0071428571428568954, 0.092500000000000025, -0.10678571428571415,
-0.0071428571428568954, 0.092500000000000025, -0.10678571428571415,
0.092500000000000025]
```

5.7. Media móvil

La media móvil (*moving average*) de orden n de una señal es una nueva señal donde cada valor es la media aritmética de los últimos n valores de la señal. Para producir los primeros n valores asumiremos que la señal tiene valor cero en todos los $n-1$ instantes de tiempo anteriores al inicial.

Elabora una función `movmean(x, n)` que devuelve la media móvil de la señal x .

```
>>> movmean([1,2,3,4,3,2,1,2,3,4,3,2,1],3)
(0.3333333333333333, 1.0, 2.0, 3.0, 3.3333333333333335, 3.0, 2.0, 1.
6666666666666667, 2.0, 3.0, 3.3333333333333335, 3.0, 2.0)
>>> movmean([1,2,3,4,3,2,1,2,3,4,3,2,1],2)
(0.5, 1.5, 2.5, 3.5, 3.5, 2.5, 1.5, 1.5, 2.5, 3.5, 3.5, 2.5, 1.5)
```